

Section 3.2 — Content Delivery Networks (CDNs) & Distributed Asset Delivery

Enhanced Study Notes (Architecture-Heavy Section)

Study Directive: Give extra weight to before/after architecture diagrams and production topology comparisons between named systems (e.g., Cloudflare vs Fastly vs AWS CloudFront, push vs pull CDN topologies, cache invalidation architectures).

Executive Overview

CDNs are the backbone of global web performance. Understanding edge hierarchies, caching strategies, and invalidation patterns separates high-scale architects from generalists. A CDN is fundamentally a globally distributed caching layer that puts content physically close to users, reducing both latency and origin load.

1. CDN Geo-Hierarchies

1.1 The Problem CDNs Solve

Before CDN (Centralized Origin):

```
graph LR; Tokyo[Tokyo User] --> Origin[Origin (Virginia)]; London[London User] --> Origin; NYC[NYC User] --> Origin;
```

Tokyo User → Origin (Virginia)
~200ms round-trip

London User → Origin (Virginia)
~110ms round-trip

NYC User → Origin (Virginia)
~10ms round-trip

Problems:

- Tokyo user: unacceptably slow
- All traffic hits Virginia → origin becomes bottleneck
- Origin bandwidth: expensive
- DDoS: single target

After CDN (Edge Cache):

Tokyo User → Tokyo PoP → (cache miss only) → Origin
~5ms cache hit ~200ms (rare)

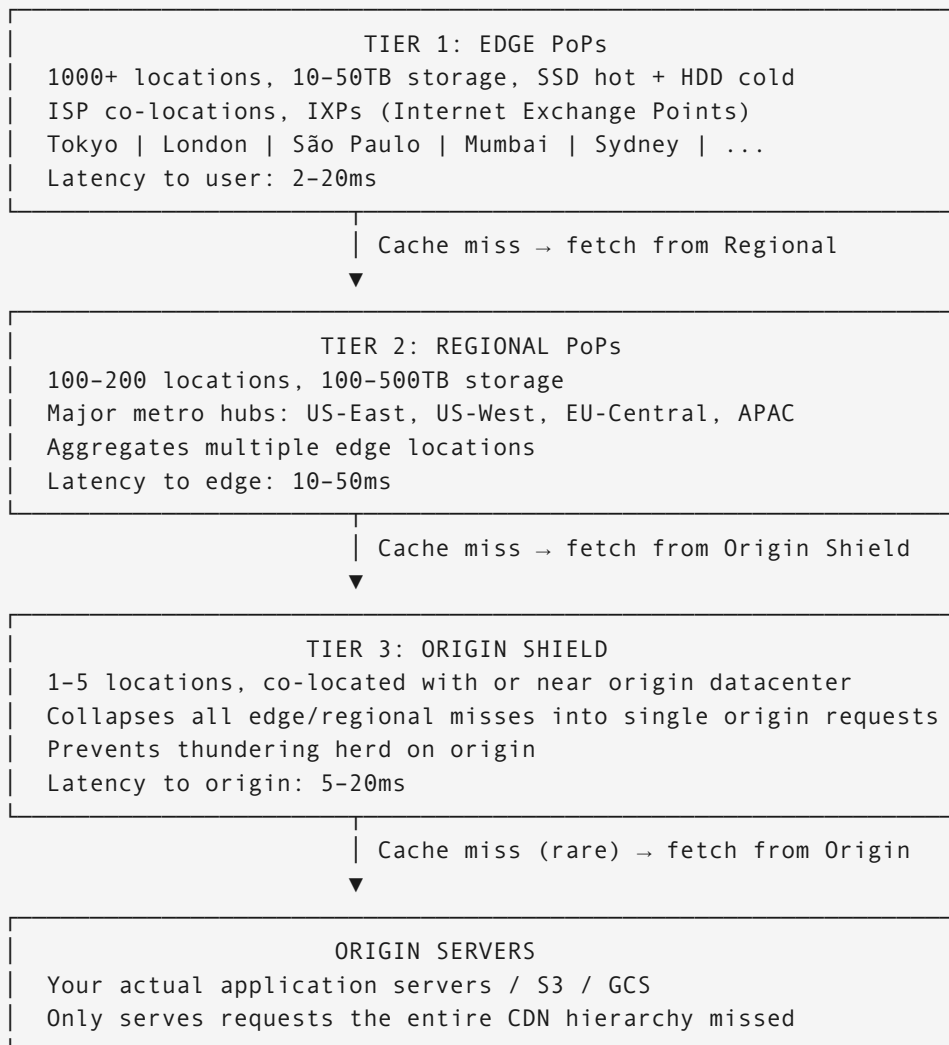
London User → London PoP → (cache miss only) → Origin
~3ms cache hit ~110ms (rare)

NYC User → NYC PoP → (cache miss only) → Origin
~1ms cache hit ~10ms (rare)

Benefits:

- 95%+ cache hit rate → origin sees 1/20 the traffic
- Users get sub-10ms latency for cached assets
- DDoS absorbed at edge (massive distributed capacity)

1.2 Three-Tier Edge Hierarchy



Production PoP Characteristics:

Tier	Count (Cloudflare)	Storage	Hit Rate
Edge	300+ cities	10–50TB SSD+HDD	~85%
Regional	~30 regions	100–500TB	~12%
Origin Shield	~5	Unlimited	~2.9%
Origin	Your infra	—	0.1%

1.3 Routing Decision Architecture

Geographic Routing Flow:

```
User DNS query: cdn.example.com
|
▼
CDN Authoritative DNS (e.g., Cloudflare, AWS Route53)
|
├─ Check: MaxMind GeoIP → user location = Tokyo
├─ Check: BGP Anycast → nearest healthy PoP = Tokyo-1
├─ Check: Health status → Tokyo-1 healthy? Yes
└─ Return: Anycast IP owned by Tokyo-1
    |
    ▼
    User connects to Tokyo-1 directly
    (BGP Anycast ensures IP is "nearby" per routing tables)

Failover:
Tokyo-1 overloaded or down →
DNS / BGP withdraws Tokyo-1 route →
User's ISP re-routes to Tokyo-2 or Osaka-1 automatically
No DNS TTL wait: BGP converges in seconds
```

Named CDN Systems Comparison:

Feature	Cloudflare	AWS CloudFront	Fastly	Akamai
PoP count	300+	450+	65+	4000+
Edge compute	Workers (V8)	Lambda@Edge	Compute@Edge (WASM)	EdgeWorkers
Cache invalidation	~150ms (instant)	1–5 min	~150ms (instant)	Minutes
Custom rules	Rules Engine	CloudFront Functions	VCL (Varnish)	EdgeSuite

Feature	Cloudflare	AWS CloudFront	Fastly	Akamai
DDoS mitigation	Built-in, free	AWS Shield	Add-on	Built-in
Best for	Small-large, free tier	AWS-native apps	Programmable edge	Enterprise, media

2. Content Ingestion Mechanics

2.1 Push vs Pull CDNs

Before (Pull CDN — Cold Start Problem):

```

Timeline for video launch at 9:00 AM:
9:00:00 — 10,000 users request video.mp4 simultaneously
9:00:00 — ALL 10,000 hit edge as cache misses (content never cached)
9:00:00 — Edge nodes fan out to regional → origin shield → origin
9:00:01 — Origin receives 10,000 simultaneous requests
9:00:05 — Origin overwhelmed, timeout errors
9:00:10 — First edge node gets content, caches it
9:00:10 — Requests 2-10,000 served from cache
Result:   First ~10 seconds: errors. Then fine.

```

After (Push CDN — Pre-Populated):

```

Timeline for video launch at 9:00 AM:
8:55:00 — Content team uploads video.mp4 to CDN origin
8:55:00 — CDN push: replicates to ALL edge nodes simultaneously
8:59:59 — All edge nodes have video.mp4 cached before launch
9:00:00 — 10,000 users request video.mp4
9:00:00 — ALL 10,000 served from edge cache (0ms origin calls)
Result:   Zero origin load. Sub-10ms responses for all users.

```

Pull CDN Architecture Detail:

```

Request flow (cache miss):
User → Edge PoP → (MISS) → Regional PoP → (MISS) → Origin Shield → Origin

Request coalescing at Edge:
1000 users request same asset simultaneously
Edge receives all 1000 requests
Edge makes EXACTLY ONE request to next tier (coalescing)
Other 999 requests wait for the one response
All 1000 get served when response arrives

```

Without coalescing:

1000 requests → 1000 origin calls → origin dies

With coalescing (Nginx proxy_cache_lock on):

1000 requests → 1 origin call → 999 queue → all served from cache

Hybrid Approach — Smart Tiering:

Content classification (ML-based):

Popularity Score = f(creator_subscribers, topic_trending,
time_since_upload, engagement_rate)

- |
 - └─ Score > 0.9 (top 1%): PUSH → all edges immediately
 - └─ Score 0.5–0.9 (next 9%): PULL + prefetch (pre-warm top regions)
 - └─ Score < 0.5 (long tail): PURE PULL (lazy, cold start acceptable)

Cost impact:

Push everything: 100% storage cost at edge

Smart tiering: ~10% storage cost, 95%+ cache hit rate

3. Cache Invalidation Vectors

3.1 The Invalidation Problem Visualized

Timeline of stale content without proper invalidation:

t=0: Origin: main.css v1 (blue background)
Edge: main.css v1 cached with TTL=24h

t=1h: Developer deploys main.css v2 (red background)
Origin: main.css v2

t=1h: User requests main.css from edge
Edge: Still serves v1 (blue background) ← STALE!
TTL has not expired. 23 hours of stale serving remain.

t=24h: Edge TTL expires → fetches v2 from origin
Finally correct. 23-hour window of stale content.

3.2 Purge Strategies Compared

Immediate Purge (Cloudflare / Fastly instant purge):

Step 1: PUT /deploy → Origin updates main.css to v2
Step 2: POST /purge {"files": ["https://cdn.example.com/main.css"]}
→ CDN receives purge request
→ Propagates purge to ALL PoPs via internal pub/sub
→ All ~300 PoPs invalidate within 150ms (Cloudflare/Fastly)
Step 3: Next request to any PoP → cache miss → fetches v2 from origin

Problem: During 150ms window, stale v1 still served.
For most applications: acceptable.
For financial data: not acceptable.

Versioned URLs (Zero-State Window):

Build step adds content hash to filename:
main.css → main.d41d8cd9.css (v1)
main.css → main.a87ff679.css (v2 after changes)

HTML links to versioned URL:
<link href="/assets/main.d41d8cd9.css"> (deploy 1)
<link href="/assets/main.a87ff679.css"> (deploy 2)

CDN behavior:
Both URLs cached simultaneously (different filenames!)
No invalidation needed (ever).
Old users on v1 get v1. New users get v2.
Stale window = 0.

Only the HTML itself needs invalidation (small, fast, tolerable).
This is how webpack, Vite, and Next.js handle static assets.

Staggered Invalidation (Progressive):

Use case: Large CDN (millions of assets), bulk deploy

Step 1: Set short TTL on old content:
Cache-Control: max-age=60, stale-while-revalidate=86400

Step 2: After deploy, let TTL expire gradually:
- Edge nodes revalidate in 60s waves
- No burst on origin (gradual traffic)
- stale-while-revalidate: serve stale while fetching new version
→ User never waits. Gets slightly stale then fresh.

Headers behavior:
t=0: TTL=60s starts
t=60: TTL expires, edge tries to revalidate
→ Serves stale to user (stale-while-revalidate)
→ Background fetch of new version
t=61: New version cached, all future requests see new version

3.3 Validation Handshakes

ETag-Based Validation:

```
Initial request (t=0):  
  Browser → Edge → Origin  
  Origin response:  
    HTTP/1.1 200 OK  
    ETag: "abc123"  
    Cache-Control: max-age=3600  
    [body: 50KB of CSS]  
  
Revalidation (t=3601, TTL expired):  
  Edge → Origin  
  GET /main.css  
  If-None-Match: "abc123"  
  
  Origin checks: has content changed?  
  If unchanged: HTTP 304 Not Modified (no body) ← saves 50KB bandwidth  
  If changed: HTTP 200 OK + ETag: "def456" + [new body]
```

Origin Shield as Invalidation Amplifier:

```
Without Origin Shield (300 PoPs, all need revalidation):  
  Asset expires simultaneously at 300 edge nodes  
  300 parallel requests hit origin  
  Origin spike: 300 QPS for 1 second  
  (Thundering herd)  
  
With Origin Shield:  
  Asset expires at 300 edge nodes  
  All 300 make ONE hop to Origin Shield (regional collapsing)  
  Origin Shield makes ONE request to origin  
  Origin sees: 1 QPS  
  Origin Shield fans back to all 300 edges  
  
Architecture:  
  300 Edges → 5 Origin Shields → 1 Origin request  
  300x reduction in origin load on cache expiry
```

3.4 Failure Modes

Cache Stampede — Numeric Example:

```
Setup: hot asset, 10,000 requests/second, TTL=1 hour  
  
t=3600: TTL expires at all edges simultaneously  
        10,000 requests/second arrive  
        ALL are cache misses (TTL just expired)
```

All 10,000 fan out to origin
Origin capacity: 100 requests/second
Result: 9,900 requests/second overload → origin crash

Mitigation 1 – Jitter:

Instead of TTL=3600, use $TTL=3600 \pm \text{random}(0, 300)$
Expiry spread over 5-minute window
Peak origin load: $10,000 / 300 \approx 33$ requests/second ← manageable

Mitigation 2 – Stale-while-revalidate:

When TTL expires, serve stale version IMMEDIATELY
Kick off background revalidation (one request to origin)
Origin receives: 1 request (not 10,000)
Once fresh content arrives, swap atomically

Mitigation 3 – Request Coalescing (Nginx proxy_cache_lock):

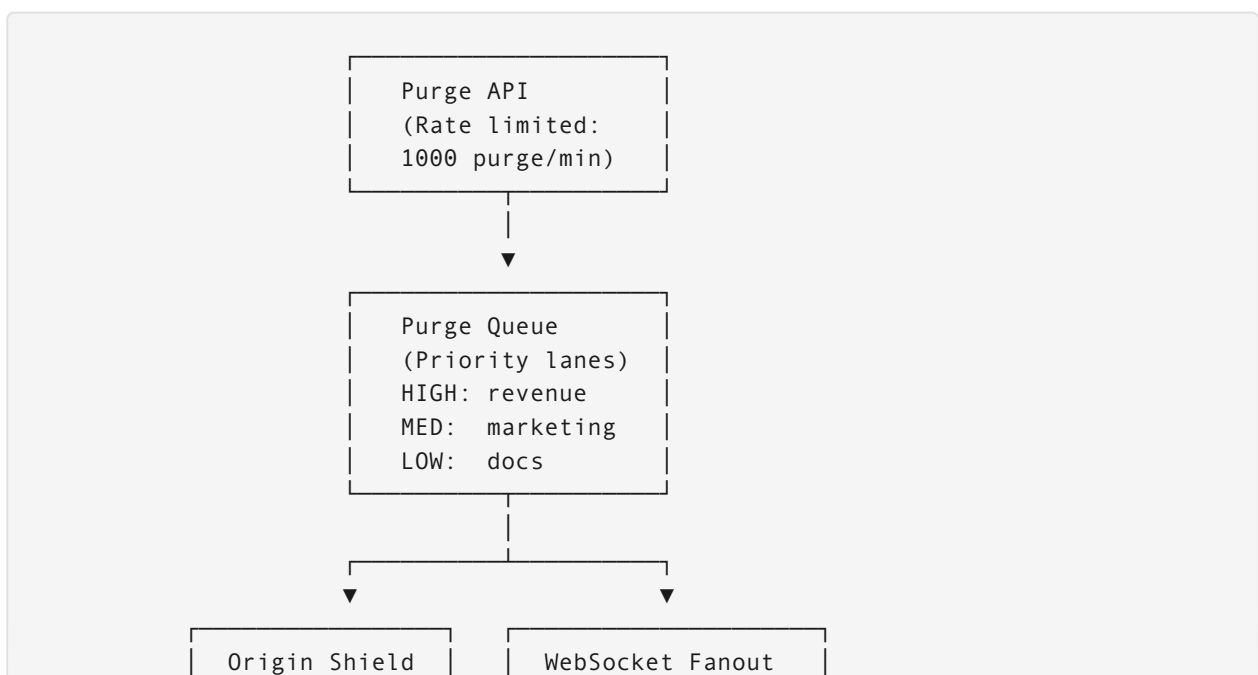
All 10,000 requests arrive simultaneously
Nginx: only 1 request sent upstream, rest queued
When upstream responds, all 10,000 served from cache
Origin: 1 request total

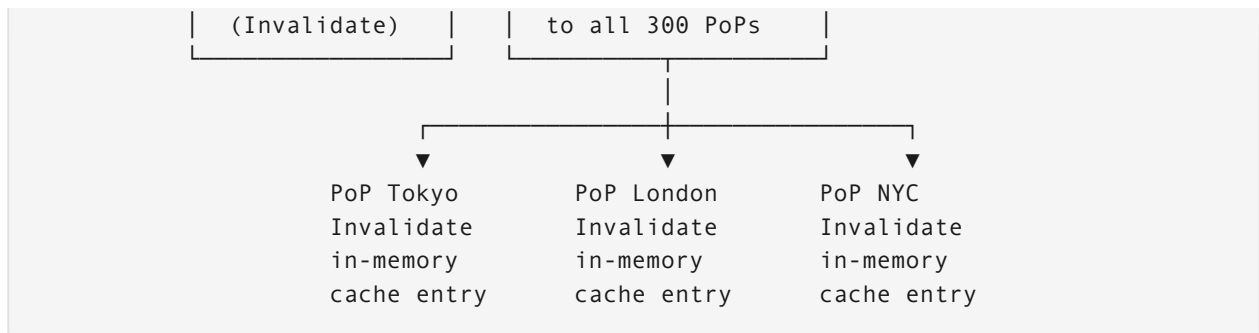
4. Interview Question — Extended Solution

Design a CDN invalidation system that guarantees no stale content is served within 1 second of purge, without overwhelming the origin.

Solution Framework

Architecture — Real-Time Purge System:





Versioned URL Strategy (Perfect Solution):

```

// Build pipeline: webpack/Vite adds content hash
// Before deploy:
//   /assets/main.css → served to old users

// After deploy:
//   /assets/main.d41d8cd9.css → new file, new URL
//   Old URL /assets/main.css → still cached, still valid
//   New HTML: <link href="/assets/main.d41d8cd9.css">

// CDN stores BOTH versions simultaneously
// Zero stale window: new users get new, old users get old
// Purge only needed for: HTML, API responses, user-generated content

// Purge only the HTML (tiny, fast):
await cdn.purge("https://example.com/index.html");
// HTML purge propagates in <1 second
// New HTML fetched → points to new hashed CSS
// Old browsers finish their sessions with old hashed CSS ← valid forever
  
```

Priority-Based Invalidation:

```

Revenue-critical assets (checkout, pricing):
  - Versioned URLs (zero stale) OR
  - Instant broadcast purge + accept <1s window

Marketing assets (homepage hero):
  - Instant broadcast purge
  - Stale-while-revalidate: 0s (no tolerance)

Long-tail assets (docs, blog):
  - Short TTL (5min) + stale-while-revalidate (1h)
  - Eventual consistency acceptable
  - Cost: no purge API call needed
  
```

Origin Protection During Mass Purge:

Scenario: Deploy affects 1M assets. All purged simultaneously.

Without protection:

1M assets expire → next request for each is cache miss → origin flood

Protection strategy:

1. Purge in waves: 10,000 assets/minute (rate limited)
2. Stale-while-revalidate: serve old version during revalidation
3. Priority queue: revenue-critical first, docs last
4. Origin Shield: all misses collapse to shield, 1 request per asset to origin
5. Circuit breaker: if origin error rate > 10%, pause purge queue

Result:

Peak origin load: 10,000 concurrent revalidations
All served stale during transition (< 1 minute staleness)
Origin never overwhelmed

Key Insight: There are two fundamentally different approaches: 1. **Versioned URLs** — zero stale window, no purge needed, requires build pipeline changes 2. **Broadcast invalidation** — acceptable for most production systems, ~150ms stale window

Most large systems use versioned URLs for static assets (JS/CSS/images) and broadcast purge for dynamic content (HTML, API responses). This combination achieves both performance and correctness.

5. Additional Interview Questions

Q2: Your CDN cache hit rate dropped from 90% to 50% overnight. How do you diagnose and fix it?

Diagnosis steps:

1. Check cache key configuration:
Did someone add a query param that busts cache?
URL: /api/data?userId=123 → unique per user → 0% hit rate
Fix: Strip userId from cache key, or don't cache this endpoint.
2. Check TTL values:
Did a deploy change Cache-Control headers to no-cache?
curl -I https://cdn.example.com/asset.js | grep cache-control
3. Check geographic distribution:
New PoP added? Fresh cold cache → local miss rate high.
Pre-warm with prefetch API call.
4. Check for cache stampede:
Did many assets expire simultaneously?
Add jitter to TTL.
5. Check content type:

New endpoints added that are dynamic (user-specific) and shouldn't be cached.
Review CDN rules: exclude /api/user/* from caching.

Fix matrix:

Query param busting: → Configure cache key ignore list
Low TTL: → Increase max-age, add stale-while-revalidate
New PoP cold: → Prefetch top 1000 assets
Dynamic content: → Add Vary: Cookie or Cache-Control: private

Q3: How would you design a CDN for a live sports broadcast serving 10M concurrent viewers?

Challenge:

Live video: cannot be cached (real-time)
But: $10\text{M users} \times 6\text{Mbps stream} = 60\text{Tbps bandwidth}$
Single origin cannot serve this

Solution – CDN-native live streaming:

Origin: 1 encoder → 1 RTMP stream → Origin PoP
Origin Shield: Receives 1 stream → replicates to 30 regional PoPs
Regional PoPs: Each serves ~333K viewers directly
Edge PoPs: Serve viewers within ~10ms radius

Segment caching for near-live:

HLS segments: 2-6 second chunks
Segments are cacheable (immutable once written)!
Only the M3U8 manifest needs real-time bypass (TTL=2s)
Segments: TTL=300s (5 min, will never change once written)

Latency stack:

Encoder → Origin Shield: ~200ms (live encoding delay)
Origin Shield → Regional: ~50ms
Regional → Edge: ~10ms
Edge → User: ~5ms
Total: ~265ms latency behind live
(Broadcast TV: 4-6 seconds. Good streaming: <3 seconds.)